

Assignment No: 10 Data Analytics III

Implement Simple Naïve Bayes classification algorithm using Python/R on iris.csv

Compute Confusion matrix to find TP, FP, TN, FN, Accuracy, Error rate, Precision. Recall on the given dataset.

In [1]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [4]:

```
data=pd.read_csv('iris.csv')
data.head()
```

Out[4]:

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

In [5]:

```
data.describe()
```

Out[5]:

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

In [6]:

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column          Non-Null Count  Dtype  
---  -
 0   sepal_length    150 non-null   float64
 1   sepal_width     150 non-null   float64
 2   petal_length    150 non-null   float64
 3   petal_width     150 non-null   float64
 4   species         150 non-null   object  
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

In [7]:

```
#check null
data.isnull().sum()
```

Out[7]:

```
sepal_length    0
sepal_width     0
petal_length    0
petal_width     0
species         0
dtype: int64
```

Step 1: Split the dataset into features and target variable

We split the data into training and testing sets to: **Short Answer: To train the model on one part (training set) and test its performance on unseen data (test set) — this helps check if the model can generalize well.**

Example: Training set: Used to teach the model. Testing set: Used to evaluate how well the model performs on new data.

Explanation: X is the feature matrix — it contains the independent variables (input features) used to predict something. Here, we are using sepal length, sepal width, petal length, and petal width as features.

Y is the target variable — it contains the dependent variable or label we want to predict. Here, it is the species of the flower (like setosa, versicolor, or virginica).

In [11]:

```
from sklearn.model_selection import train_test_split
X = data[['sepal_length', 'sepal_width', 'petal_length', 'petal_width']] # here we dropped the class which has species
Y = data['species'] #selected species
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.3, random_state=42)
```

In [12]:

```
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

classifier = GaussianNB()

classifier.fit(X_train, y_train)
```

Out[12]:

▼ GaussianNB ?

GaussianNB()

In [13]:

```
from sklearn import metrics

y_pred = classifier.predict(X_test)

print("Accuracy Score: ", metrics.accuracy_score(y_test, y_pred)*100)
```

Accuracy Score: 97.77777777777777

Confusion Matrix (Short Explanation for Viva) A confusion matrix is a table used to evaluate the performance of a classification model. It helps to find:

Predicted Positive Predicted Negative

Actual Positive True Positive (TP) False Negative (FN) Actual Negative False Positive (FP) True Negative (TN)

- **1. Accuracy Formula:** $(TP + TN) / (TP + FP + TN + FN)$ **Meaning:** It tells how often the model is correct overall both when it predicts positive and negative correctly. "Out of all predictions, how many were correct?"
- **2. Error Rate Formula:** $(FP + FN) / (TP + FP + TN + FN)$ **Meaning:** It is the opposite of accuracy it tells how often the model was wrong. "Out of all predictions, how many were incorrect?"
- **3. Precision Formula:** $TP / (TP + FP)$ **Meaning:** It tells how many of the predicted positives are actually correct. "When the model says positive, how much can we trust it?"
- **4. Recall (Sensitivity) Formula:** $TP / (TP + FN)$ **Meaning:** It tells how many actual positives were correctly identified. "How well the model catches all the true positives?"

In [14]:

```
from sklearn.metrics import confusion_matrix
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score

print("\nOverall")
conf_matrix = confusion_matrix(y_test, y_pred)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='macro')
recall = recall_score(y_test, y_pred, average='macro')
error_rate = 1 - accuracy
print("Confusion Matrix:\n", conf_matrix)
print("Accuracy:", accuracy)
print("Error Rate:", error_rate)
print("Precision:", precision)
print("Recall:", recall)
```

```
Overall
Confusion Matrix:
[[19  0  0]
 [ 0 12  1]
 [ 0  0 13]]
Accuracy: 0.9777777777777777
Error Rate: 0.022222222222222254
Precision: 0.9761904761904763
Recall: 0.9743589743589745
```

In [18]:

```
# Classification report
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	19
versicolor	1.00	0.92	0.96	13
virginica	0.93	1.00	0.96	13
accuracy			0.98	45
macro avg	0.98	0.97	0.97	45
weighted avg	0.98	0.98	0.98	45

In [20]:

```
cm = confusion_matrix(y_test, y_pred)
TP = cm[0][0]

FN = cm[0][1] + cm[0][2]

FP = cm[1][0] + cm[2][0]

TN = cm[1][1] + cm[1][2] + cm[2][1] + cm[2][2]

print("TP = ", TP)
```

```
print("FN = ", FN)
print("FP = ", FP)
print("TN = ", TN)
```

```
TP = 19
FN = 0
FP = 0
TN = 26
```

```
In [ ]:
```